SUBJECT :- ANDROID APPLICATIONS

CHAPTER-1 INRTODUCTION TO ANDROID

What is Android?

Android is a software package and linux based operating system for mobile devices such as tablet computers and smartphones.

It is developed by Google and later the OHA (Open Handset Alliance). Java language is mainly used to write the android code even though other languages can be used.

The goal of android project is to create a successful real-world product that improves the mobile experience for end users.

There are many code names of android such as Lollipop, Kitkat, Jelly Bean, Ice cream Sandwich, Froyo, Ecliar, Donut etc which is covered in next page.

What is Open Handset Alliance (OHA)

It's a consortium of 84 companies such as google, samsung, AKM, synaptics, KDDI, Garmin, Teleca, Ebay, Intel etc.

It was established on 5th November, 2007, led by Google. It is committed to advance open standards, provide services and deploy handsets using the Android Plateform.

Features of Android:-

After learning what is android, let's see the features of android. The important features of android are given below:

1) It is open-source.

2) Anyone can customize the Android Platform.

3) There are a lot of mobile applications that can be chosen by the consumer.

4) It provides many interesting features like weather details, opening screen, live RSS (Really Simple Syndication) feeds etc.

It provides support for messaging services(SMS and MMS), web browser, storage (SQLite), connectivity (GSM, CDMA, Blue Tooth, Wi-Fi etc.), media, handset layout etc.

Categories of Android applications:-

There are many android applications in the market. The top categories are:

- Entertainment
- Tools
- Communication

- Productivity
- Personalization
- Music and Audio
- Social
- Media and Video
- Travel and Local etc.

Dalvik Virtual Machine | DVM

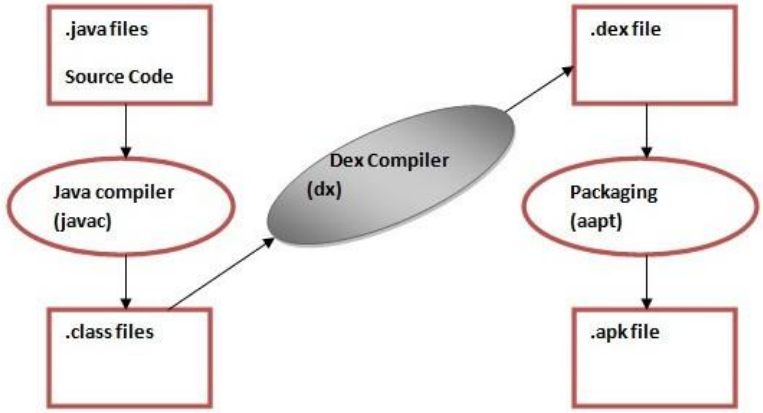
As we know the modern JVM is high performance and provides excellent memory management. But it needs to be optimized for low-powered handheld devices as well.

The **Dalvik Virtual Machine (DVM)** is an android virtual machine optimized for mobile devices. It optimizes the virtual machine for *memory*, *battery life* and *performance*.

Dalvik is a name of a town in Iceland. The Dalvik VM was written by Dan Bornstein.

The Dex compiler converts the class files into the .dex file that run on the Dalvik VM. Multiple class files are converted into one dex file.

Let's see the compiling and packaging process from the source file:



The **javac tool** compiles the java source file into the class file.

The **dx tool** takes all the class files of your application and generates a single .dex file. It is a platform-specific tool.

The Android Assets Packaging Tool (aapt) handles the packaging process.

File Extension: .apk

An APK file is the file format used to install the applications on android operating system. A program in android is first compiled, and then all of its parts are packaged into one single file to make it an APK file. APK files can be saved in compressed zip format, with any decompression or unzip tool it can be opened. You can either change the extension to .zip or open the file directly using unzip tools. They are a type of archive file. When you are downloading and installing an android application then actually you are installing an APK file. You can install an APK file directly from a desktop, or from within a file manager app without downloading. By

default, ability to install from desktop, or file manager app is disabled for security reasons. TO enable it, you have to change the setting of 'Unknown sources' from settings.

Activity:-

An activity is a class that represents a single screen. It is like a Frame in AWT. An activity represents a single screen with a user interface just like window or frame of Java.Android activity is the subclass of ContextThemeWrapper class. Android system initiates its program with in an **Activity** starting with a call on *onCreate()* callback method. There is a sequence of callback methods that start up an activity and a sequence of callback methods that tear down an activity

The Activity class defines the following call backs i.e. events. You don't need to implement all the callbacks methods.

1. **onCreate**():-This is the first callback and called when the activity is first created.

2. **onStart**():- This callback is called when the activity becomes visible to the user.

3. **onResume**():-This is called when the user starts interacting with the application.

4. **onPause**():- The paused activity does not receive user input and cannot execute any code and called when the current activity is being paused and the previous activity is being resumed.

5. **onStop**():- This callback is called when the activity is no longer visible.

6. **onDestroy**():- This callback is called before the activity is destroyed by the system.

7. **onRestart**():- This callback is called when the activity restarts after stopping it.

Service:- A **service** is a component that runs in the background to perform long-running operations without needing to interact with the user and it works even if application is destroyed. A service can essentially take two states –

There are two types of services local and remote. Local service is accessed from within the application whereas remote service is accessed remotely from other applications running on the same device.

1. **Started**:- A service is started when an application component, such as an activity, starts it by calling **startService**(). Once started, a service can run in the background indefinitely, even if the component that started it is destroyed.

2. **Bound**:- A service is bound when an application component binds to it by calling **bindService**(). A bound service offers a client-server interface that allows components to interact with the service, send requests, get results, and even do so across processes with interprocess communication (IPC).

To create an service, you create a Java class that extends the Service base class or one of its existing subclasses. The **Service** base class defines various callback methods and the most important are given below.

1. **onStartCommand**():- The system calls this method when another component, such as an activity, requests that the service be started, by calling **startService**(). If you implement this method, it is your responsibility to stop the service when its work is done, by calling **stopSelf**() or **stopService**() methods.

2. **onBind**():- The system calls this method when another component wants to bind with the service by calling **bindService**(). If you implement this method, you must provide an interface that clients use to communicate with the service, by returning an IBinder object. You must always implement this method, but if you don't want to allow binding, then you should return null.

3. **onUnbind**():- The system calls this method when all clients have disconnected from a particular interface published by the service.

4. **onRebind**():- The system calls this method when new clients have connected to the service, after it had previously been notified that all had disconnected in its onUnbind(Intent).

5. **onCreate**():- The system calls this method when the service is first created using **onStartCommand**() or **onBind**(). This call is required to perform one-time set-up.

6. **onDestroy**():- The system calls this method when the service is no longer used and is being destroyed. Your service should implement this to clean up any resources such as threads, registered listeners, receivers, etc.

Broadcast Receivers:- Broadcast Receivers simply respond to broadcast messages from other applications or from the system itself. These messages are sometime called events or intents. There are following two important steps to make BroadcastReceiver works for the system broadcasted intents −

Creating the Broadcast Receiver.
Registering Broadcast Receiver

There is one additional steps in case you are going to implement your custom intents then you will have to create and broadcast those intents.

Creating the Broadcast Receiver

A broadcast receiver is implemented as a subclass of **BroadcastReceiver** class and overriding the **onReceive**() method where each message is received as a **Intent** object parameter.

```
public class MyReceiver extends BroadcastReceiver {

  @Override

  public void onReceive(Context context, Intent intent) {

    Toast.makeText(context, "Intent Detected.", Toast.LENGTH_LONG).show();

  }

}
```

Registering Broadcast Receiver

An application listens for specific broadcast intents by registering a broadcast receiver in *AndroidManifest.xml* file. Consider we are going to register MyReceiver for system generated event ACTION_BOOT_COMPLETED which is fired by the system once the Android system has completed the boot process.

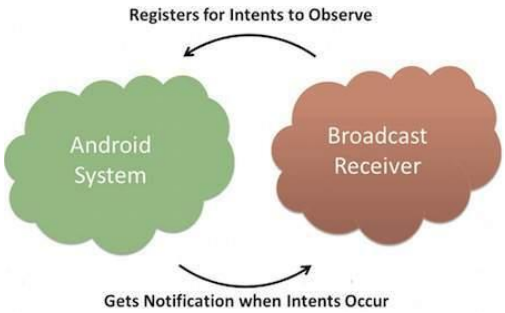


Fig:- Broadcast-Receiver

```
<application

  android:icon="@drawable/ic_launcher"

  android:label="@string/app_name"

  android:theme="@style/AppTheme" >

  <receiver android:name="MyReceiver">



    <intent-filter>

      <action android:name="android.intent.action.BOOT_COMPLETED">

      </action>

    </intent-filter>

  </receiver>

</application>
```

Now whenever your Android device gets booted, it will be intercepted by BroadcastReceiver *MyReceiver* and implemented logic inside ***onReceive()*** will be executed.

There are several system generated events defined as final static fields in the Intent class. The following table lists a few important system events.

1. android.intent.action.BATTERY_CHANGED

Sticky broadcast containing the charging state, level, and other information about the battery.

2. android.intent.action.BATTERY_LOW

Indicates low battery condition on the device.

3. android.intent.action.BATTERY_OKAY

Indicates the battery is now okay after being low.

4. android.intent.action.BOOT_COMPLETED

This is broadcast once, after the system has finished booting.

5. android.intent.action.BUG_REPOR

Show activity for reporting a bug.

6. android.intent.action.CALL

Perform a call to someone specified by the data.

7. android.intent.action.CALL_BUTTON

The user pressed the "call" button to go to the dialer or other appropriate UI for placing a call.

8. android.intent.action.DATE_CHANGED

The date has changed.

9. android.intent.action.REBOOT

Have the device reboot.

Content Provider:- Content Providers are used to share data between the applications. A content provider component supplies data from one application to others on request. Such requests are handled by the methods of the ContentResolver class. A content provider can use different ways to store its data and the data can be stored in a database, in files, or even over a network.

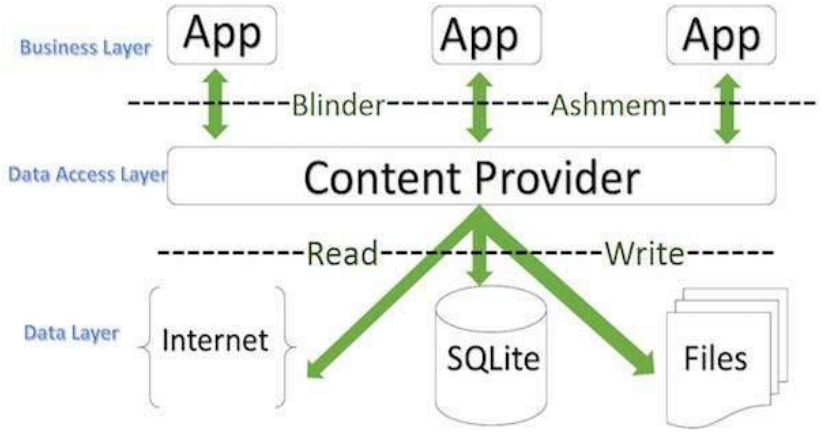


Fig:-ContentProvider

sometimes it is required to share data across applications. This is where content providers become very useful.

A content provider is implemented as a subclass of ContentProvider class and must implement a standard set of APIs that enable other applications to perform transactions.

```
public class My Application extends  ContentProvider {

}
```

Content URIs

To query a content provider, you specify the query string in the form of a URI which has following format –

<prefix>://<authority>/<data_type>/<id>

Here is the detail of various parts of the URI –

1. **prefix**:- This is always set to content://

2. **authority**:- This specifies the name of the content provider, for example contacts, browser etc. For third-party content providers, this could be the fully qualified name, such as com.tutorialspoint.statusprovider

3. **data_type**:- This indicates the type of data that this particular provider provides. For example, if you are getting all the contacts from the Contacts content provider, then the data path would be people and URI would look like thiscontent://contacts/people

4. **id**:- This specifies the specific record requested. For example, if you are looking for contact number 5 in the Contacts content provider then URI would look like this content://contacts/people/5.

Create Content Provider

This involves number of simple steps to create your own content provider.

First of all you need to create a Content Provider class that extends the **ContentProviderbaseclass**.

Second, you need to define your content provider URI address which will be used to access the content.

Next you will need to create your own database to keep the content. Usually, Android uses SQLite database and framework needs to override onCreate() method which will use SQLite Open Helper method to create or open the provider's database. When your application is launched, the onCreate() handler of each of its Content Providers is called on the main application thread.

Next you will have to implement Content Provider queries to perform different database specific operations.

Finally register your Content Provider in your activity file using <provider> tag.

Here is the list of methods which you need to override in Content Provider class to have your Content Provider working –

onCreate() :- This method is called when the provider is started.

query():- This method receives a request from a client. The result is returned as a Cursor object.

insert():- This method inserts a new record into the content provider.

delete():- This method deletes an existing record from the content provider.

update():- This method updates an existing record from the content provider.

getType():- This method returns the MIME type of the data at the given URI.

UI Components:-

View:- The basic building block for user interface is a **View** object which is created from the View class and occupies a rectangular area on the screen and is responsible for drawing and event handling. View is the base class for widgets, which are used to create interactive UI components like buttons, text fields, etc.

The **ViewGroup** is a subclass of **View** and provides invisible container that hold other Views or other ViewGroups and define their layout properties.

At third level we have different layouts which are subclasses of ViewGroup class and a typical layout defines the visual structure for an Android user interface and can be created either at run time using **View/ViewGroup** objects or you can declare your layout using simple XML file **main_layout.xml** which is located in the res/layout folder of your project.

Notification:- A notification is a message you can display to the user outside of your application's normal UI. When you tell the system to issue a notification, it first appears as an icon in the notification area. To see the details of the notification, the user opens the notification drawer. Both the notification area and the notification drawer are system-controlled areas that the user can view at any time.

Intents and Filters:-

An Android Intent is an abstract description of an operation to be performed. It can be used with startActivity to launch an Activity, broadcastIntent to send it to any interested BroadcastReceiver components, and startService(Intent) or bindService(Intent, ServiceConnection, int) to communicate with a background Service.

The intent itself, an Intent object, is a passive data structure holding an abstract description of an operation to be performed.

Types of Intents

There are following two types of intents supported by Android

Explicit Intents

Explicit intent going to be connected internal world of application,suppose if you wants to connect one activity to another activity, we can do this quote by explicit intent, below image is connecting first activity to second activity by clicking button.

These intents designate the target component by its name and they are typically used for application-internal messages - such as an activity starting a subordinate service or launching a sister activity. For example −

```
// Explicit Intent by specifying its class name

Intent i = new Intent(FirstActivity.this, SecondActivity.class);

// Starts TargetActivity

startActivity(i);
```

Implicit Intents

These intents do not name a target and the field for the component name is left blank. Implicit intents are often used to activate components in other applications. For example −

```
Intent read1=new Intent();

read1.setAction(android.content.Intent.ACTION_VIEW);

read1.setData(ContactsContract.Contacts.CONTENT_URI);

startActivity(read1);
```

Android versions, name, and API level

Code name	Version numbers	API level	Release date
No codename	1.0	1	September 23, 2008
No codename	1.1	2	February 9, 2009
Cupcake	1.5	3	April 27, 2009
Donut	1.6	4	September 15, 2009
Eclair	2.0 - 2.1	5 - 7	October 26, 2009
Froyo	2.2 - 2.2.3	8	May 20, 2010
Gingerbread	2.3 - 2.3.7	9 - 10	December 6, 2010
Honeycomb	3.0 - 3.2.6	11 - 13	February 22, 2011
Ice Cream Sandwich	4.0 - 4.0.4	14 - 15	October 18, 2011

Jelly Bean	4.1 - 4.3.1	16 - 18	July 9, 2012
KitKat	4.4 - 4.4.4	19 - 20	October 31, 2013
Lollipop	5.0 - 5.1.1	21- 22	November 12, 2014
Marshmallow	6.0 - 6.0.1	23	October 5, 2015
Nougat	7.0	24	August 22, 2016
Nougat	7.1.0 - 7.1.2	25	October 4, 2016
Oreo	8.0	26	August 21, 2017
Oreo	8.1	27	December 5, 2017
Pie	9.0	28	August 6, 2018
Android 10	10.0	29	September 3, 2019
Android 11	11	30	September 8, 2020

CHAPTER-2 Environment Setup and Basic Project Structure

AndroidManifest.xml file

The AndroidManifest.xml file *contains information of your package*, including components of the application such as activities, services, broadcast receivers, content providers etc.

It performs some other tasks also:

- It is **responsible to protect the application** to access any protected parts by providing the permissions.
- It also **declares the android api** that the application is going to use.
- It **lists the instrumentation classes**. The instrumentation classes provides profiling and other informations. These informations are removed just before the application is published etc.

This is the required xml file for all the android application and located inside the root directory.

Elements of the AndroidManifest.xml file

The elements used in the above xml file are described below.

<manifest>

manifest is the root element of the AndroidManifest.xml file. It has **package** attribute that describes the package name of the activity class.

<application>

application is the subelement of the manifest. It includes the namespace declaration. This element contains several subelements that declares the application component such as activity etc.

The commonly used attributes are of this element are **icon**, **label**, **theme** etc.

android:icon represents the icon for all the android application components.

android:label works as the default label for all the application components.

android:theme represents a common theme for all the android activities.

<activity>

activity is the subelement of application and represents an activity that must be defined in the AndroidManifest.xml file. It has many attributes such as label, name, theme, launchMode etc.

android:label represents a label i.e. displayed on the screen.

android:name represents a name for the activity class. It is required attribute.

<intent-filter>

intent-filter is the sub-element of activity that describes the type of intent to which activity, service or broadcast receiver can respond to.

<action>

It adds an action for the intent-filter. The intent-filter must have at least one action element.

<category>

It adds a category name to an intent-filter.

Gradle:- Gradle is a build system (open source) that is used to automate building, testing, deployment, etc. "build.gradle" are scripts where one can automate the tasks. For example, the simple task to copy some files from one directory to another can be performed by the Gradle build script before the actual build process happens.

Why is Gradle Needed?

Every Android project needs a Gradle for generating an apk from the *.java* and *.xml* files in the project. Simply put, a gradle takes all the source files (java and XML) and applies appropriate tools, e.g., converts the java files into dex files and compresses all of them into a single file known as apk that is actually used. There are two types of build.gradle scripts .

Top-level build.gradle:

It is located in the root project directory and its main function is to define the build configurations that will be applied to all the modules in the project.

Module-level build.gradle:

Located in the *project/module* directory of the project this Gradle script is where all the dependencies are defined and where the SDK versions are declared. This script has many functions in the project which include additional build types and override settings in the main/app manifest or *top-level build.gradle* file

Resources:-

Directory & Resource Type

1. **anim**/:- XML files that define property animations. They are saved in res/anim/ folder and accessed from the R.anim class.

2. **color**/:- XML files that define a state list of colors. They are saved in res/color/ and accessed from the R.color class.

3. **drawable**/:- Image files like .png, .jpg, .gif or XML files that are compiled into bitmaps, state lists, shapes, animation drawable. They are saved in res/drawable/ and accessed from the R.drawable class.

4.**layout**/:- XML files that define a user interface layout. They are saved in res/layout/ and accessed from the R.layout class.

5. **menu**/:- XML files that define application menus, such as an Options Menu, Context Menu, or Sub Menu. They are saved in res/menu/ and accessed from the R.menu class.

6. **raw**/:- Arbitrary files to save in their raw form. You need to call Resources.openRawResource() with the resource ID, which is R.raw.filename to open such raw files.

7. **values**/:- XML files that contain simple values, such as strings, integers, and colors. For example, here are some filename conventions for resources you can create in this directory −

arrays.xml for resource arrays, and accessed from the R.array class.

integers.xml for resource integers, and accessed from the R.integer class.

bools.xml for resource boolean, and accessed from the R.bool class.

colors.xml for color values, and accessed from the R.color class.

dimens.xml for dimension values, and accessed from the R.dimen class.

strings.xml for string values, and accessed from the R.string class.

styles.xml for styles, and accessed from the R.style class.

8. **xml**/:- Arbitrary XML files that can be read at runtime by calling Resources.getXML(). You can save various configuration files here which will be used at run time.

Android R.java file

Android R.java is *an auto-generated file by aapt* (Android Asset Packaging Tool) that contains resource IDs for all the resources of res/ directory.

If you create any component in the activity_main.xml file, id for the corresponding component is automatically created in this file. This id can be used in the activity source file to perform any action on the component.

Assets provide a way to add arbitrary files like text, XML, HTML, fonts, music, and video in the application. If one tries to add these files as "**resources**", Android will treat them into its resource system and you will be unable to get the raw data. If one wants to access data untouched, Assets are one way to do it.

Drawable resources:- A drawable resource is a general concept for a graphic that can be drawn to the screen and which you can retrieve with APIs such as getDrawable(int) or apply to another XML resource with attributes such as android:drawable and android:icon. There are several different types of drawables:

Bitmap File:- A bitmap graphic file (.png, .webp, .jpg, or .gif). Creates a BitmapDrawable.

Nine-Patch File:- A PNG file with stretchable regions to allow image resizing based on content (.9.png). Creates a NinePatchDrawable.

Layer List:- A Drawable that manages an array of other Drawables. These are drawn in array order, so the element with the largest index is be drawn on top. Creates a LayerDrawable.

State List:- An XML file that references different bitmap graphics for different states (for example, to use a different image when a button is pressed). Creates a StateListDrawable.

Level List:- An XML file that defines a drawable that manages a number of alternate Drawables, each assigned a maximum numerical value. Creates a LevelListDrawable.

Transition Drawable:- An XML file that defines a drawable that can cross-fade between two drawable resources. Creates a TransitionDrawable.

Inset Drawable:- An XML file that defines a drawable that insets another drawable by a specified distance. This is useful when a View needs a background drawable that is smaller than the View's actual bounds.

Clip Drawable:- An XML file that defines a drawable that clips another Drawable based on this Drawable's current level value. Creates a ClipDrawable.

Scale Drawable:- An XML file that defines a drawable that changes the size of another Drawable based on its current level value. Creates a ScaleDrawable

Shape Drawable:- An XML file that defines a geometric shape, including colors and gradients. Creates a GradientDrawable.

Android UI Layouts

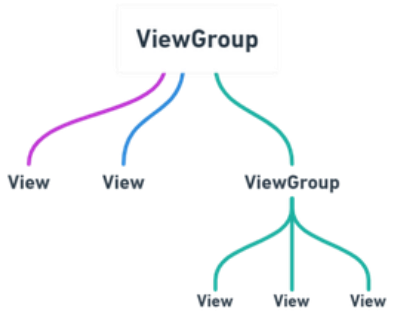
Android **Layout** is used to define the user interface that holds the UI controls or widgets that will appear on the screen of an android application or activity screen. Generally, every application is a combination of View and ViewGroup. As we know, an android application contains a large number of activities and we can say each activity is one page of the application. So, each activity contains multiple user interface components and

those components are the instances of the View and ViewGroup. All the elements in a layout are built using a hierarchy of **View** and **ViewGroup** objects.

View

A **View** is defined as the user interface which is used to create interactive UI components such as TextView, ImageView, EditText, RadioButton, etc., and is responsible for event handling and drawing. They are Generally Called Widgets.

View:- A **ViewGroup** act as a base class for layouts and layouts parameters that hold other Views or ViewGroups and to define the layout properties. They are Generally Called layouts.



ViewGroup

The Android framework will allow us to use UI elements or widgets in two ways:

Use UI elements in the XML file

Create elements in the Kotlin file dynamically

Types of Android Layout

Android Linear Layout: LinearLayout is a ViewGroup subclass, used to provide child View elements one by one either in a particular direction either horizontally or vertically based on the orientation property.
Android Relative Layout: RelativeLayout is a ViewGroup subclass, used to specify the position of child View elements relative to each other like (A to the right of B) or relative to the parent (fix to the top of the parent).
Android Constraint Layout: ConstraintLayout is a ViewGroup subclass, used to specify the position of layout constraints for every child View relative to other views present. A ConstraintLayout is similar to a RelativeLayout, but having more power.
Android Frame Layout: FrameLayout is a ViewGroup subclass, used to specify the position of View elements it contains on the top of each other to display only a single View inside the FrameLayout.
Android Table Layout: TableLayout is a ViewGroup subclass, used to display the child View elements in rows and columns.
Android Web View: WebView is a browser that is used to display the web pages in our activity layout.
Android ListView: ListView is a ViewGroup, used to display scrollable lists of items in a single column.

Android Grid View: GridView is a ViewGroup that is used to display a scrollable list of items in a grid view of rows and columns.

Android Emulator

The **Android emulator** is an **Android Virtual Device (AVD),** which represents a specific Android device. We can use the Android emulator as a target device to execute and test our Android application on our PC. The Android emulator provides almost all the functionality of a real device. We can get the incoming phone calls and text messages. It also gives the location of the device and simulates different network speeds. Android emulator simulates rotation and other hardware sensors. It accesses the Google Play store, and much more

logcat

The Android logging system provides a mechanism for collecting and viewing system debug output. Logs from various applications and portions of the system are collected in a series of circular buffers, which then can be viewed and filtered by the logcat command. You can use logcat from an ADB shell to view the log messages.

Syntax

[adb] logcat [<option>] ... [<filter-spec>] ...

You can run logcat as an adb command or directly in a shell prompt of your emulator or connected device. To view log output using adb, navigate to your SDK platform-tools/ directory and execute:

$ adb logcat

You can create a shell connection to a device and execute:

$adbshell
logcat

Introduction to DDMS

The Dalvik Debug Monitor Service (DDMS) is a debugging tool used in the Android platform. The Dalvik Debug Monitor Service is downloaded as part of the Android SDK. Some of the services provided by the DDMS are port forwarding, on-device screen capture, on-device thread and heap monitoring, and radio state information.

For example, by using the DDMS' LogCat feature, developers can view log messages regarding the state of the application and the device. LogCat can pinpoint the exact line number on which an error occurred.

Another DDMS feature, known as the Emulator Control, allows developers to simulate phone states and activities. For example, it can simulate different types of networks such as GPRS, EDGE, and UTMS, which can have different network characteristics such as speed and latency.

Aside from the features mentioned earlier, the Dalvik Debug Monitor Service also provides LogCat, process, and radio state information as well as incoming call, SMS, and location data spoofing.

This debugging tool can be integrated into the Eclipse IDE by adding the ADT (Android Development Tools) plug-in. Otherwise, it can be accessed from the command line and will automatically connect to any running emulator.

Run apps on the Android Emulator

The Android Emulator simulates Android devices on your computer so that you can test your application on a variety of devices and Android API levels without needing to have each physical device.

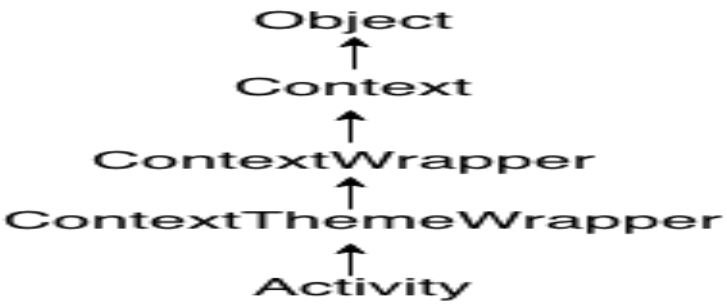
The emulator provides almost all of the capabilities of a real Android device. You can simulate incoming phone calls and text messages, specify the location of the device, simulate different network speeds, simulate rotation and other hardware sensors, access the Google Play Store, and much more.

Testing your app on the emulator is in some ways faster and easier than doing so on a physical device. For example, you can transfer data faster to the emulator than to a device connected over USB.

The emulator comes with predefined configurations for various Android phone, tablet, Wear OS, and Android TV devices.

CHAPTER-3 Android Fundaments and User Interface Design

Android Activity Lifecycle



Android Activity Lifecycle is controlled by 7 methods of android.app.Activity class. The android Activity is the subclass of ContextThemeWrapper class.

An activity is the single screen in android. It is like window or frame of Java.

By the help of activity, you can place all your UI components or widgets in a single screen.

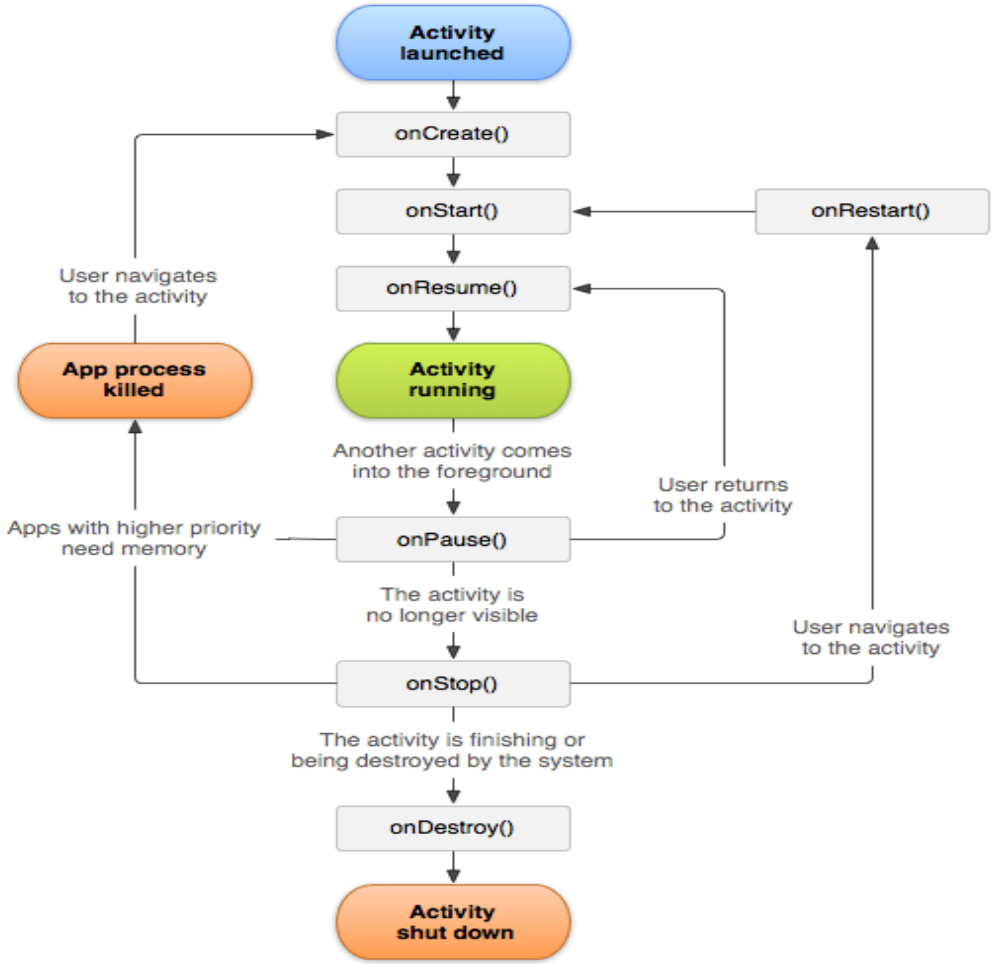
The 7 lifecycle method of Activity describes how activity will behave at different states.

Android Activity Lifecycle methods

Let's see the 7 lifecycle methods of android activity.

Method	Description
onCreate	called when activity is first created.
onStart	called when activity is becoming visible to the user.

onResume	called when activity will start interacting with the user.
onPause	called when activity is not visible to the user.
onStop	called when activity is no longer visible to the user.
onRestart	called after your activity is stopped, prior to start.
onDestroy	called before the activity is destroyed.



Basic UI Components

TextView is the user interface which displays the text message on the screen to the user. It is based on the layout size, style, and color, etc. TextView is used to set and display the text according to our specifications.

Class Syntax:

public class TextView

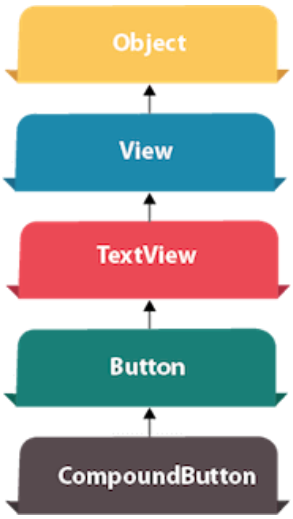
extends View

implements ViewTreeObserver.OnPreDrawListener

Syntax:

```
<SomeLayout>
    .
    .
    <TextView>
        android:SomeAttribute1 = "Value of attribute1"
        android:SomeAttribute2 = "Value of attribute2"
        .
        .
        android:SomeAttributeN = "Value of attributeN"
    </TextView>
    .
    .
</SomeLayout>
```

Android Button



Android Button represents a push-button. The android.widget.Button is subclass of TextView class and CompoundButton is the subclass of Button class.

There are different types of buttons in android such as RadioButton, ToggleButton, CompoundButton etc.

RadioButton

RadioButton is a two states button which is either checked or unchecked. If a single radio button is unchecked, we can click it to make checked radio button. Once a radio button is checked, it cannot be marked as unchecked by user.

RadioButton is generally used with *RadioGroup*. RadioGroup contains several radio buttons, marking one radio button as checked makes all other radio buttons as unchecked.

A **EditText** is an overlay over TextView that configures itself to be editable. It is the predefined subclass of TextView that includes rich editing capabilities.

EditText Attributes

Following are the important attributes related to EditText control. You can check Android official documentation for complete list of attributes and related methods which you can use to change these attributes are run time.

Inherited from android.widget.TextView Class –

1. **android:autoText:-** If set, specifies that this TextView has a textual input method and automatically corrects some common spelling errors.

2. **android:drawableBottom**:- This is the drawable to be drawn below the text.

3. **android:drawableRight**:- This is the drawable to be drawn to the right of the text.

4. **android:editable**:- If set, specifies that this TextView has an input method.

5. **android:text**:- This is the Text to display.

Inherited from android.view.View Class –

1. **android:background**:- This is a drawable to use as the background.

2. **android:contentDescription**:- This defines text that briefly describes content of the view.

3. **android:id**:- This supplies an identifier name for this view.

4. **android:onClick**:- This is the name of the method in this View's context to invoke when the view is clicked.

5. **android:visibility**:- This controls the initial visibility of the view.

ImageView

ImageView class is used to display any kind of image resource in the android application either it can be android.graphics.Bitmap or android.graphics.drawable.Drawable (it is a general abstraction for anything that can be drawn in Android). ImageView class or android.widget.ImageView inherits the android.view.View class which is the subclass of Kotlin.Any class. Application of ImageView is also in applying tints to an image in

order to reuse a drawable resource and create overlays on background images. Moreover, ImageView is also used to control the size and movement of an image.

Android CheckBox

Android CheckBox is a type of two state button either checked or unchecked.

There can be a lot of usage of checkboxes. For example, it can be used to know the hobby of the user, activate/deactivate the specific action etc.

Android CheckBox class is the subclass of CompoundButton class.

Android CheckBox class

The android.widget.CheckBox class provides the facility of creating the CheckBoxes.

Methods of CheckBox class

There are many inherited methods of View, TextView, and Button classes in the CheckBox class. Some of them are as follows:

public boolean isChecked():- Returns true if it is checked otherwise false.

public void setChecked(boolean status):- Changes the state of the CheckBox.

Android ProgressBar

We can display the android progress bar dialog box to display the status of work being done e.g. downloading file, analyzing status of work etc.

Android ProgressDialog is the subclass of AlertDialog class.

The ProgressDialog class provides methods to work on progress bar like setProgress(), setMessage(), setProgressStyle(), setMax(), show() etc. The progress range of Progress Dialog is 0 to 10000.

Event Handling:- **Events are a useful way to collect data about a user's interaction with interactive components of Applications.** Like button presses or screen touch etc. The Android framework maintains an event queue as first-in, first-out (FIFO) basis. You can capture these events in your program and take appropriate action as per requirements.

There are following three concepts related to Android Event Management −

- **Event Listeners** − An event listener is an interface in the View class that contains a single callback method. These methods will be called by the Android framework when the View to which the listener has been registered is triggered by user interaction with the item in the UI.
- **Event Listeners Registration** − Event Registration is the process by which an Event Handler gets registered with an Event Listener so that the handler is called when the Event Listener fires the event.
- **Event Handlers** − When an event happens and we have registered an event listener for the event, the event listener calls the Event Handlers, which is the method that actually handles the event.

Event Listeners & Event Handlers

Event Handler	Event Listener & Description
onClick()	**OnClickListener()** This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. You will use onClick() event handler to handle such event.
onLongClick()	**OnLongClickListener()** This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. for one or more seconds. You will use onLongClick() event handler to handle such event.
onFocusChange()	**OnFocusChangeListener()** This is called when the widget looses its focus ie. user goes away from the view item. You will use onFocusChange() event handler to handle such event.
onKey()	**OnFocusChangeListener()** This is called when the user is focused on the item and presses or releases a hardware key on the device. You will use onKey() event handler to handle such event.
onTouch()	**OnTouchListener()** This is called when the user presses the key, releases the key, or any movement gesture on the screen. You will use onTouch() event handler to handle such event.
onMenuItemClick()	**OnMenuItemClickListener()** This is called when the user selects a menu item. You will use onMenuItemClick() event handler to handle such event.
onCreateContextMenu()	**onCreateContextMenuItemListener()** This is called when the context menu is being built(as the result of a sustained "long click)

Layouts:- Android LinearLayout is a view group that aligns all children in either vertically or horizontally.

LinearLayout Attributes

Following are the important attributes specific to LinearLayout –

1. **android:id**:- This is the ID which uniquely identifies the layout.

2. **android:baselineAligned**:- This must be a boolean value, either "true" or "false" and prevents the layout from aligning its children's baselines.

3. **android:baselineAlignedChildIndex**:- When a linear layout is part of another layout that is baseline aligned, it can specify which of its children to baseline align.

4. **android:divider**:- This is drawable to use as a vertical divider between buttons. You use a color value, in the form of "#rgb", "#argb", "#rrggbb", or "#aarrggbb".

5. **android:gravity**:- This specifies how an object should position its content, on both the X and Y axes. Possible values are top, bottom, left, right, center, center_vertical, center_horizontal etc.

6. **android:orientation**:- This specifies the direction of arrangement and you will use "horizontal" for a row, "vertical" for a column. The default is horizontal.

7. **android:weightSum**:-Sum up of child weight

Relative Layout

RelativeLayout enables you to specify how child views are positioned relative to each other. The position of each view can be specified as relative to sibling elements or relative to the parent.

RelativeLayout Attributes

Following are the important attributes specific to RelativeLayout −

1. **android:id**:- This is the ID which uniquely identifies the layout.

2. **android:gravity**:- This specifies how an object should position its content, on both the X and Y axes. Possible values are top, bottom, left, right, center, center_vertical, center_horizontal etc.

3. **android:ignoreGravity**:- This indicates what view should not be affected by gravity.

Frame Layout:- Frame Layout is designed to block out an area on the screen to display a single item. Generally, FrameLayout should be used to hold a single child view, because it can be difficult to

organize child views in a way that's scalable to different screen sizes without the children overlapping each other.You can, however, add multiple children to a FrameLayout and control their position within the FrameLayout by assigning gravity to each child, using the android:layout_gravity attribute.

FrameLayout Attributes

Following are the important attributes specific to FrameLayout −

1. **android:id**:- This is the ID which uniquely identifies the layout.

2. **android:foreground**:- This defines the drawable to draw over the content and possible values may be a color value, in the form of "#rgb", "#argb", "#rrggbb", or "#aarrggbb".

3. **android:foregroundGravity**:- Defines the gravity to apply to the foreground drawable. The gravity defaults to fill. Possible values are top, bottom, left, right, center, center_vertical, center_horizontal etc.

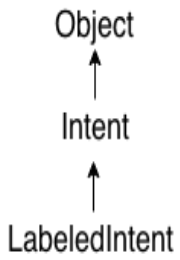
4. **android:measureAllChildren**:- Determines whether to measure all children or just those in the VISIBLE or INVISIBLE state when measuring. Defaults to false.

Intent:- Android Intent is the *message* that is passed between components such as activities, content providers, broadcast receivers, services etc.

It is generally used with startActivity() method to invoke activity, broadcast receivers etc.

The dictionary meaning of intent is *intention or purpose*. So, it can be described as the intention to do action.

The LabeledIntent is the subclass of android.content.Intent class.



Android intents are mainly used to:

- Start the service
- Launch an activity
- Display a web page
- Display a list of contacts
- Broadcast a message
- Dial a phone call etc.

Types of Android Intents

There are two types of intents in android: implicit and explicit.

1) Implicit Intent:- Implicit Intent doesn't specifiy the component. In such case, intent provides information of available components provided by the system that is to be invoked.

For example, you may write the following code to view the webpage.

```
1. Intent intent=new Intent(Intent.ACTION_VIEW);
2. intent.setData(Uri.parse("http://www.javatpoint.com"));
3. startActivity(intent);
```

2) **Explicit Intent**:- Explicit Intent specifies the component. In such case, intent provides the external class to be invoked.

```
1. Intent i = new Intent(getApplicationContext(), ActivityTwo.class);
2. startActivity(i);
```

CHAPTER-4 Android Menus

In android, Menu is an important part of UI component which is used to provide some common functionality around the application. With the help of menu user can experience smooth and consistent experience throughout the application.

In order to use menu, we should define it in separate XML file and use that file in our application based on our requirements. Also, we can use menu APIs to represent user actions and other options in our android application activities.

Android Different Types of Menus

In android, we have a three types of Menus available to define a set of options and actions in our android applications.

The Menus in android applications are following –

Android Options Menu – Android Options Menu is a primary collection of menu items in an android application and useful for actions that have a global impact on the searching application.

Android Context Menu – Android Context Menu is a floating menu only appears when user click for a long time on an element and useful for elements that effect the selected content or context frame.

Android Popup Menu – Android Popup Menu displays a list of items in a vertical list which presents to the view that invoked the menu and useful to provide an overflow of actions that related to specific content.

Shared Prefernces:- One of the most Interesting Data Storage options **Android** provides its users is **Shared Preferences**. **Shared Preferences** is the way in which one can store and retrieve small amounts of primitive

data as key/value pairs to a file on the device storage such as String, int, float, Boolean that make up your preferences in an XML file inside the app on the device storage. **Shared Preferences** can be thought of as a dictionary or a key/value pair.

How to Create Shared Preferences?

The first thing we need to do is to create one shared preferences file per app. So name it with the package name of your app- unique and easy to associate with the app. When you want to get the values, call the **getSharedPreferences()** method. Shared Preferences provide modes of storing the data (private mode and public mode). It is for backward compatibility- use only **MODE_PRIVATE** to be secure.

public abstract SharedPreferences getSharedPreferences (String name, int mode)

*This method takes two arguments, the first being the name of the **SharedPreference(SP) file** and the other is the **context mode** that we want to store our file in.*

***MODE_PUBLIC** will make the file public which could be accessible by other applications on the device*

***MODE_PRIVATE** keeps the files private and secures the user's data.*

***MODE_APPEND** is used while reading the data from the SP file.*

Nested classes of Shared Preferences

1. **SharedPreferences.Editor**: Interface used to write(edit) data in the SP file. Once editing has been done, one must **commit()** or **apply()** the changes made to the file.
2. **SharedPreferences.OnSharedPreferenceChangeListener()**: Called when a shared preference is changed, added, or removed. This may be called even if a preference is set to its existing value. This callback will be run on your main thread.

Following are the methods of Shared Preferences

1. **contains(String key)**: This method is used to check whether the preferences contain a preference.

2. **edit()**: This method is used to create a new Editor for these preferences, through which you can make modifications to the data in the preferences and atomically commit those changes back to the SharedPreferences object.

3. **getAll()**: This method is used to retrieve all values from the preferences.

4. **getBoolean(String key, boolean defValue)**: This method is used to retrieve a boolean value from the preferences.

5. **getFloat(String key, float defValue)**: This method is used to retrieve a float value from the preferences.

6. **getInt(String key, int defValue)**: This method is used to retrieve an int value from the preferences.

7. **getLong(String key, long defValue)**: This method is used to retrieve a long value from the preferences.

8. **getString(String key, String defValue)**: This method is used to retrieve a String value from the preferences.

9. **getStringSet(String key, Set defValues)**: This method is used to retrieve a set of String values from the preferences.

10. **registerOnSharedPreferencechangeListener(SharedPreferences.OnSharedPreferencechangeListener listener)**: This method is used to registers a callback to be invoked when a change happens to a preference.

11. **unregisterOnSharedPreferencechangeListener(SharedPreferences.OnSharedPreferencechangeListener listener)**: This method is used to unregisters a previous callback.

CHAPTER-5 Advanced UI Components

Date and Time in Android are formatted using the SimpleDateFormat library from Java, using Calendar instance which helps to get the current system date and time. The current date and time are of the type Long which can be converted to a human-readable date and time.

Android ListView is a view which groups several items and display them in vertical scrollable list. The list items are automatically inserted to the list using an Adapter that pulls content from a source such as an array or database.

An adapter actually bridges between UI components and the data source that fill data into UI Component. Adapter holds the data and send the data to adapter view, the view can takes the data from adapter view and shows the data on different views like as spinner, list view, grid view etc.

The ListView and GridView are subclasses of AdapterView and they can be populated by binding them to an Adapter, which retrieves data from an external source and creates a View that represents each data entry.

1	**android:id** This is the ID which uniquely identifies the layout.
2	**android:divider** This is drawable or color to draw between list items.
3	**android:dividerHeight** This specifies height of the divider. This could be in px, dp, sp, in, or mm.
4	**android:entries** Specifies the reference to an array resource that will populate the ListView.

5	**android:footerDividersEnabled** When set to false, the ListView will not draw the divider before each footer view. The default value is true.
6	**android:headerDividersEnabled** When set to false, the ListView will not draw the divider after each header view. The default value is true.

Grid View:- A GridView is a type of AdapterView that displays items in a two-dimensional scrolling grid. Items are inserted into this grid layout from a database or from an array. The adapter is used for displaying this data, **setAdapter()** method is used to join the adapter with GridView. The main function of the adapter in GridView is to fetch data from a database or array and insert each piece of data in an appropriate item that will be displayed in GridView. This is how the GridView structure looks like. Note that we are going to implement this project using the **Java** language.

XML Attributes of GridView

- **android:numColumns**: This attribute of GridView will be used to decide the number of columns that are to be displayed in Grid.
- **android:horizontalSpacing**: This attribute is used to define the spacing between two columns of GridView.
- **android:verticalSpacing**: This attribute is used to specify the spacing between two rows of GridView.

Card View-: CardView is a new widget in Android that can be used to display any sort of data by providing a rounded corner layout along with a specific elevation. CardView is the view that can display views on top of each other. The main usage of CardView is that it helps to give a rich feel and look to the UI design. This widget can be easily seen in many different Android Apps. CardView can be used for creating items in listview or inside Recycler View. The best part about CardView is that it extends Framelayout and it can be displayed on all platforms of Android.

RecyclerView is a ViewGroup added to the android studio as a successor of the GridView and ListView. It is an improvement on both of them and can be found in the latest v-7 support packages. It has been created to make possible construction of any lists with **XML** layouts as an item which can be customized vastly while *improving on the efficiency of ListViews and GridViews*. This improvement is achieved by recycling the views which are out of the visibility of the user. For example, if a user scrolled down to a position where items 4 and 5 are visible; items 1, 2, and 3 would be cleared from the memory to reduce memory consumption.

Implementation: To implement a basic RecyclerView three sub-parts are needed to be constructed which offer the users the degree of control they require in making varying designs of their choice.

1. **The Card Layout:** The card layout is an XML layout which will be treated as an item for the list created by the RecyclerView.

2. **The ViewHolder:** The ViewHolder is a java class that stores the reference to the card layout views that have to be dynamically modified during the execution of the program by a list of data obtained either by online databases or added in some other way.
3. **The Data Class:** The Data class is a custom java class that acts as a structure for holding the information for every item of the RecyclerView.

Adapter:- An adapter acts like a bridge between a data source and the user interface. It reads data from various data sources, coverts it into View objects and provide it to the linked Adapter view to create UI components.The data source or dataset can be an Array object, a List object etc.You can create your own Adapter class by extending the BaseAdapter class, which is the parent class for all other adapter class. Android SDK also provides some ready-to-use adapter classes, such as ArrayAdapter, SimpleAdapter etc.

ArrayAdapter is the most commonly used adapter in android. When you have a list of single type items which are stored in an array you can use ArrayAdapter. Likewise, if you have a list of phone numbers, names, or cities. ArrayAdapter has a layout with a single TextView. If you want to have a more complex layout instead of ArrayAdapter use **CustomArrayAdapter**. The basic syntax for **ArrayAdapter is given as:**
public ArrayAdapter(Context context, int resource, int textViewResourceId, T[] objects)

Parameters
context: It is used to pass the reference of the current class. Here 'this' keyword is used to pass the current class reference. Instead of 'this' we could also use the getApplicationContext() method which is used for the Activity and the **getApplication()** method which is used for Fragments.
public ArrayAdapter(this, int resource, int textViewResourceId, T[] objects)

resource: It is used to set the layout file(.xml files) for the list items.
public ArrayAdapter(this, R.layout.itemListView, int textViewResourceId, T[] objects)

textViewResourceId: It is used to set the TextView where you want to display the text data.
public ArrayAdapter(this, R.layout.itemListView, R.id.itemTextView, T[] objects)

objects: These are the array object which is used to set the array element into the TextView.
String courseList[] = {"C-Programming", "Data Structure", "Database", "Python",
"Java", "Operating System","Compiler Design", "Android Development"};
ArrayAdapter arrayAdapter = new ArrayAdapter(this, R.layout.itemListView, R.id.itemTextView, courseList[]);

BaseAdapter:-

- BaseAdapter is a common base class of a general implementation of an Adapter that can be used in ListView, GridView, Spinner etc.
- Whenever you need a customized list in a ListView or customized grids in a GridView you create your own adapter and extend base adapter in that.
- Base Adapter can be extended to create a custom Adapter for displaying a custom list item.

Dialogs

A dialog is a small window that prompts the user to make a decision or enter additional information. A dialog does not fill the screen and is normally used for modal events that require users to take an action before they can proceed.

The Dialog class is the base class for dialogs, but you should avoid instantiating Dialog directly. Instead, use one of the following subclasses:

AlertDialog**:-** A dialog that can show a title, up to three buttons, a list of selectable items, or a custom layout.

DatePickerDialog **or** TimePickerDialog**:-**A dialog with a pre-defined UI that allows the user to select a date or time.

Toast:- Andorid Toast can be used to display information for the short period of time. A toast contains message to be displayed quickly and disappears after sometime.

The android.widget.Toast class is the subclass of java.lang.Object class.

You can also create custom toast as well for example toast displaying image. You can visit next page to see the code for custom toast.

Toast class

Toast class is used to show notification for a particular interval of time. After sometime it disappears. It doesn't block the user interaction.

Constants of Toast class

There are only 2 constants of Toast class which are given below.

public static final int LENGTH_LONG :- displays view for the long duration of time.

public static final int LENGTH_SHORT:- displays view for the short duration of time.

Methods of Toast class

The widely used methods of Toast class are given below.

public static Toast makeText(Context context, CharSequence text, int duration):- makes the toast **containing** text and duration.

public void show():-displays toast.

public void setMargin (float horizontalMargin, float verticalMargin):- changes the horizontal and vertical margin difference.

Android Popup Menu: Android Popup Menu displays a list of items in a vertical list which presents to the view that invoked the menu and useful to provide an overflow of actions that related to specific content. A **PopupMenu** displays a Menu in a popup window anchored to a View. The popup will be shown below the anchored View if there is room(space) otherwise above the View. If any IME(Input Method Editor) is

visible the popup will not overlap it until the View(to which the popup is anchored) is touched. Touching outside the popup window will dismiss it.

Android Fragments

Android Fragment is the part of activity, it is also known as sub-activity. There can be more than one fragment in an activity. Fragments represent multiple screen inside one activity.

Android fragment lifecycle is affected by activity lifecycle because fragments are included in activity.

Each fragment has its own life cycle methods that is affected by activity life cycle because fragments are embedded in activity.

The FragmentManager class is responsible to make interaction between fragment objects.

Android Life Cycle Methods

1)	onAttach(Activity)	it is called only once when it is attached with activity.
2)	onCreate(Bundle)	It is used to initialize the fragment.
3)	onCreateView(LayoutInflater, ViewGroup, Bundle)	creates and returns view hierarchy.
4)	onActivityCreated(Bundle)	It is invoked after the completion of onCreate() method.
5)	onViewStateRestored(Bundle)	It provides information to the fragment that all the saved state of fragment view hierarchy has been restored.
6)	onStart()	makes the fragment visible.
7)	onResume()	makes the fragment interactive.
8)	onPause()	is called when fragment is no longer interactive.
9)	onStop()	is called when fragment is no longer visible.
10)	onDestroyView()	allows the fragment to clean up resources.
11)	onDestroy()	allows the fragment to do final clean up of fragment state.
12)	onDetach()	It is called immediately prior to the fragment no longer being associated with its activity.

Material Design:- Material Design Components (MDC Android) offers designers and developers a way to implement Material Design in their Android application. Developed by a core team of engineers and UX designers at Google, these components enable a reliable development workflow to build beautiful and functional Android applications. Material design in Android is one of the key features that attracts and engages the customer towards the application. This is a special type of design, which is guided by Google

Floating Button:- The floating action button is a bit different button from the ordinary buttons. Floating action buttons are implemented in the app's UI for primary actions (promoted actions) for the users and the actions under the floating action button are prioritized by the developer. For example the actions like adding an item in the existing list.

Types of Floating Action Button

There are mainly **four** types of floating action buttons available on Android.

- Normal/Regular Floating Action Button
- Mini Floating Action Button
- Extended Floating Action Button
- Theming Floating Action Button

Toolbar:- A Toolbar is similar to an Actionbar and can be a complete replacement to Android Actionbar. The Toolbar was initially introduced in Android Lollipop version. Along with the release of Android Lollipop and material design, Android has also updated the *AppCompat Support Libraries* so that we can use Toolbars on devices running API Level 7 and above (Android versions 2.1.x and above).

CHAPTER-6 THREADS IN ANDROID

UI Thread:- UI Thread allows your tasks to do background work and then move the results to UI elements such as bitmaps.Every app has its own special thread that runs UI objects such as View objects, This thread is called the UI thread. Only objects running on the UI thread have access to other objects on that thread. Because tasks that you run on a thread from a thread pool *aren't* running on your UI thread, they don't have access to UI objects. To move data from a background thread to the UI thread, use a Handler that's running on the UI thread.

Worker thread:

Worker threads are background threads. They are the threads that are created separately, other than the UI thread. Since blocking the UI thread is restricted according to the rule, the user should run the child processes and tasks in worker threads.

An example of the creation and working of worker thread is given below:-

Public void onClick(View v) { new Thread(new Runnable() {

public void run() {

Bitmap b = loadImageFromNetwork("http://example.com/image.png");

mImageView.setImageBitmap(b);}

}).start(); }

Handlers:- It will be complicated to manage communication with all these threads if you are managing a large group of threads. So, Android has provided handlers to make the inter-process communication easier.

A handler allows you to send and process Message and Runnable objects associated with a thread's MessageQueue. Each handler instance is associated with a single thread and that thread's message queue.

This class is responsible for enqueuing any task to the message queue and processing it.

When you create a new handler, it is bound to the thread/message queue of the thread that is creating it — from that point on, it will deliver messages and runnables to that message queue and executes them as they come out of the message queue.

In simple words, we can say that a handler is a component that can be attached to a thread and then made to perform an action on that thread via simple messages or Runnable tasks.

It works in conjunction with another component, looper, which is in charge of message processing in a particular thread.

When a handler is created, it can get a Looper object in the constructor, which indicates which thread the handler is attached to. If you want to use a handler attached to the main thread, you need to use the looper associated with the main thread by calling Looper.getMainLooper().

There are two main usages for a handler:

1. To schedule messages and runnables to be executed at some point in the future.
2. To enqueue an action to be performed on a different thread than your own.

What is Runnable?

In java the interface **Runnable** in basically an abstraction for an executable command.
We talked about the Thread class as a low level concurrency construct for java and android.
Well another building block is the `Runnable`, an interface that comes from the Java API, and meant to specify and encapsulate code that is intended to be executed by a Java thread instance or any other class that handles this `Runnable`.

It's an interface and is defined in the `java.lang` package and has an abstract method called `run()` that we have to implement.

Usage

The Runnable interface is used extensively to execute code in Threads. The Runnable interface has one abstract method called `run()`:

public abstract void run ()

Normally you implement that method whenever your class implements Runnable interface. The method will then get called to start executing the active part of the class' code.

```
public class MyRunnable implements Runnable {

   public void run(){

      Log.d("Generic","Running in the Thread " +

      Thread.currentThread().getId());

   }

}
```

With that, our `Runnable` subclass can now be passed to `Thread` and is executed independently in the concurrent line of execution:

Async Task:- Android AsyncTask going to do background operation on background thread and update on main thread. In android we cant directly touch background thread to main thread in android development. asynctask help us to make communication between background thread to main thread.

Methods of AsyncTask

- **onPreExecute()** – Before doing background operation we should show something on screen like progressbar or any animation to user. we can directly comminicate background operation using on doInBackground() but for the best practice, we should call all asyncTask methods .
- **doInBackground(Params)** – In this method we have to do background operation on background thread. Operations in this method should not touch on any mainthread activities or fragments.
- **onProgressUpdate(Progress…)** – While doing background operation, if you want to update some information on UI, we can use this method.
- **onPostExecute(Result)** – In this method we can update ui of background operation result.

Generic Types in Async Task

- **TypeOfVarArgParams** – It contains information about what type of params used for execution.
- **ProgressValue** – It contains information about progress units. While doing background operation we can update information on ui using onProgressUpdate().
- **ResultValue** –It contains information about result type.

CHAPTER- 7 NOTIFICATION & SERVICES

Broadcast Receivers:- Broadcast in android is the system-wide events that can occur when the device starts, when a message is received on the device or when incoming calls are received, or when a device goes to airplane mode, etc. Broadcast Receivers are used to respond to these system-wide events. Broadcast Receivers allow us to register for the system and application events, and when that event happens, then the register receivers get notified. There are mainly two types of Broadcast Receivers:

- **Static Broadcast Receivers:** These types of Receivers are declared in the manifest file and works even if the app is closed.
- **Dynamic Broadcast Receivers:** These types of receivers work only if the app is active or minimized.

There are two ways to register your broadcast receiver: statically in the manifest, or dynamically in your activity.

Static registration

To register your broadcast receiver statically, add a <receiver> element to your AndroidManifest.xml file. Within the <receiver> element:

- Use the path to your BroadcastReceiver subclass as the android:name attribute.
- To prevent other applications from sending broadcasts to your receiver, set the optional android:exported attribute to false. This is an important security guideline.
- To specify the types of intents the component is listening for, use a nested <intent-filter> element.

```
<receiver
android:name="com.example.myproject.AlarmReceiver"
android:exported="false">
<intent-filter>
  <action android:name="com.example.myproject.intent.action.ACTION_SHOW_TOAST"/>
</intent-filter>
</receiver>
```

Dynamic registration and unregistration

You can also register a broadcast receiver dynamically, which ties its operation to the lifecycle of your activity. To register your receiver dynamically, call registerReceiver() and pass in the BroadcastReceiver object and an intent filter. For example:

```
IntentFilter intentFilter = new IntentFilter();

intentFilter.addAction(ACTION_SHOW_TOAST);



mReceiver = new AlarmReceiver();

registerReceiver(mReceiver, intentFilter);
```

Android Notification

Android Notification provides short, timely information about the action happened in the application, even it is not running. The notification displays the icon, title and some amount of the content text.

Set Android Notification Properties

The properties of Android notification are set using **NotificationCompat.Builder** object. Some of the notification properties are mention below:

- **setSmallIcon()**: It sets the icon of notification.
- **setContentTitle()**: It is used to set the title of notification.
- **setContentText()**: It is used to set the text message.
- **setAutoCancel()**: It sets the cancelable property of notification.
- **setPriority()**: It sets the priority of notification.

Android Service Tutorial

Android service is a component that is *used to perform operations on the background* such as playing music, handle network transactions, interacting content providers etc. It doesn't has any UI (user interface).

The service runs in the background indefinitely even if application is destroyed.

Moreover, service can be bounded by a component to perform interactivity and inter process communication (IPC).

The android.app.Service is subclass of ContextWrapper class.

Life Cycle of Android Service

There can be two forms of a service.The lifecycle of service can follow two different paths: started or bound.

1) Started Service

A service is started when component (like activity) calls **startService()** method, now it runs in the background indefinitely. It is stopped by **stopService()** method. The service can stop itself by calling the **stopSelf()** method.

2) Bound Service

A service is bound when another component (e.g. client) calls **bindService()** method. The client can unbind the service by calling the **unbindService()** method.

The service cannot be stopped until all clients unbind the service.

CHAPTER- 8 Storage and Content Provider

Data storage in Android

Android provides several options for you to save persistent application data. The solution you choose depends on your specific needs, such as whether the data should be private to your application or accessible to other applications (and the user) and how much space your data requires.

Your data storage options are the following:

Using Shared Preferences

The SharedPreferencesclass provides a general framework that allows you to save and retrieve persistent key-value pairs of primitive data types. You can use SharedPreferencesto save any

primitive data: booleans, floats, ints, longs, and strings. This data will persist across user sessions (even if your application is killed).

User Preferences

Shared preferences are not strictly for saving "user preferences," such as what ringtone a user has chosen. If you're interested in creating user preferences for your application, see PreferenceActivity, which provides an Activity framework for you to create user preferences, which will be automatically persisted (using shared preferences).

To get a SharedPreferencesobject for your application, use one of two methods:

- **getSharedPreferences()** – Use this if you need multiple preferences files identified by name, which you specify with the first parameter.
- **getPreferences()** – Use this if you need only one preferences file for your Activity. Because this will be the only preferences file for your Activity, you don't supply a name.

To write values:

1. Call edit()to get a SharedPreferences.Editor.
2. Add values with methods such as putBoolean()and putString().
3. Commit the new values with commit()

To read values, use SharedPreferencesmethods such as getBoolean()and getString().

Using the Internal Storage

You can save files directly on the device's internal storage. By default, files saved to the internal storage are private to your application and other applications cannot access them (nor can the user). When the user uninstalls your application, these files are removed.

To create and write a private file to the internal storage:

1. Call openFileOutput()with the name of the file and the operating mode. This returns a FileOutputStream.
2. Write to the file with write().
3. Close the stream with close().

For example:

```
String FILENAME = "hello_file";
String string = "hello world!";

FileOutputStream fos = openFileOutput(FILENAME, Context.MODE_PRIVATE);
fos.write(string.getBytes());
fos.close();
```

MODE_PRIVATEwill create the file (or replace a file of the same name) and make it private to your application. Other modes available are: MODE_APPEND, MODE_WORLD_READABLE, and MODE_WORLD_WRITEABLE.

To read a file from internal storage:

1. Call openFileInput()and pass it the name of the file to read. This returns a FileInputStream.
2. Read bytes from the file with read().
3. Then close the stream with close().

Other useful methods

getFilesDir():- Gets the absolute path to the filesystem directory where your internal files are saved.

getDir():- Creates (or opens an existing) directory within your internal storage space.

deleteFile():- Deletes a file saved on the internal storage.

fileList():- Returns an array of files currently saved by your application.

Using the External Storage

Every Android-compatible device supports a shared "external storage" that you can use to save files. This can be a removable storage media (such as an SD card) or an internal (non-removable) storage. Files saved to the external storage are world-readable and can be modified by the user when they enable USB mass storage to transfer files on a computer.

Using a Network Connection

You can use the network (when it's available) to store and retrieve data on your own web-based services. To do network operations, use classes in the following packages:

- java.net.*
- android.net.*

SQLite is another data storage available in Android where we can store data in the user's device and can use it any time when required. In this article, we will take a look at creating an SQLite database in the Android app and adding data to that database in the Android app. SQLite Database is an open-source database provided in Android which is used to store data inside the user's device in the form of a Text file. We can perform so many operations on this data such as adding new data, updating, reading, and deleting this data. SQLite is an offline database that is locally stored in the user's device and we do not have to create any connection to connect to this database.

How Data is Being Stored in the SQLite Database?

Data is stored in the SQLite database in the form of **tables**. When we stored this data in our SQLite database it is arranged in the form of tables that are similar to that of an excel sheet. Below is the representation of our SQLite database which we are storing in our SQLite database.

Important Methods in SQLite Database

Below is the several important methods that we will be using in this SQLite database integration in Android.

getColumnNames()	This method is used to get the Array of column names of our SQLite table.
getCount()	This method will return the number of rows in the cursor.
isClosed()	This method returns a Boolean value when our cursor is closed.
getColumnCount()	This method returns the total number of columns present in our table.
getColumnName(int columnIndex)	This method will return the name of the column when we passed the index of our column in it.
getColumnIndex(String columnName)	This method will return the index of our column from the name of the column.
getPosition()	This method will return the current position of our cursor in our table.

CRUD is nothing but an abbreviation for the basic operations that we perform in any database. And the operations are

- **Create**
- **Read**
- **Update**
- **Delete**

Creating the Table

```
CREATE TABLE employees (
    id INTEGER NOT NULL CONSTRAINT employees_pk PRIMARY KEY AUTOINCREMENT,
    name varchar(200) NOT NULL,
    department varchar(200) NOT NULL,
    joiningdate datetime NOT NULL,
    salary double NOT NULL
);
```

Creating a new Record

INSERT INTO employees

(name, department, joiningdate, salary) VALUES ('Belal Khan', 'Technical', '2017-09-30 10:00:00', '40000');

Reading All Existing Records

SELECT * FROM employees;

Reading Specific Record

SELECT * FROM employees WHERE id = 1;

Updating a Record

UPDATE employees

SET

name = 'Belal Haque',

department = 'Research and Development',

salary = '100000'

WHERE id = 1;

Deleting a Record

DELETE FROM employees WHERE id = 1;

Content Provider:- A content provider component supplies data from one application to others on request. Such requests are handled by the methods of the ContentResolver class. A content provider can use different ways to store its data and the data can be stored in a database, in files, or even over a network. Content providers let you centralize content in one place and have many different applications access it as needed. A content provider behaves very much like a database where you can query it, edit its content, as well as add or delete content using insert(), update(), delete(), and query() methods. In most cases this data is stored in an **SQlite** database.

A content provider is implemented as a subclass of **ContentProvider** class and must implement a standard set of APIs that enable other applications to perform transactions.

public class My Application extends ContentProvider {

}